

Sección 8: Resiliencia y Límites de la Sincronía en Arquitecturas Distribuidas

→ ¿Si falla alguno de los micros?

Resiliencia y Límites de la Sincronía en Arquitecturas Distribuidas

- **Tecnología:** Resilience4j en Spring Boot 4.
- **Patrones Implementados:**
 - **Circuit Breaker:** Protección contra fallos totales (Inventory caído -> Circuito Abierto).
 - **Retry Pattern:** Estrategia de reintentos con *Exponential Backoff* para fallos transitorios.
- **El Experimento "TimeLimiter":**
 - Se implementó TimeLimiter para controlar latencia.
 - **Descubrimiento Crítico:** En arquitectura bloqueante (MVC), el timeout libera al usuario pero deja un **"Hilo Zombie"** en el servidor, causando inconsistencia de datos (Stock descontado, Orden fallida).
- **Refactorización Final:**
 - Se eliminó @TimeLimiter y CompletableFuture del flujo síncrono placeOrder para limpiar la arquitectura.
 - **Conclusión:** La sincronía HTTP tiene un límite duro para la consistencia transaccional. La solución real requiere mensajería.

109. Cuando los microservicios fallan (Circuit breaker)

Hasta ahora, todo funcionaba

El sistema que hemos construido asume un mundo ideal

ASUNCIÓN 1

Inventory responde

Siempre rápido.
Siempre disponible.

ASUNCIÓN 2

La BD no se cae

Las consultas
siempre retornan.

ASUNCIÓN 3

La red es rápida

Sin latencia.
Sin paquetes perdidos.

Este es el "Happy Path": el escenario donde nada sale mal. Es útil para desarrollar rápido, pero es una mentira en producción.

La realidad es otra cosa

Esto no es hipótesis. Es lo que pasa todos los días en sistemas reales.

El escenario

Un usuario confirma un pedido en **Order-Service**.

Order-Service necesita chequear stock, así que llama a **Inventory-Service**.

Pero **Inventory-Service** **no responde**. O peor aún: responde en 30 segundos.

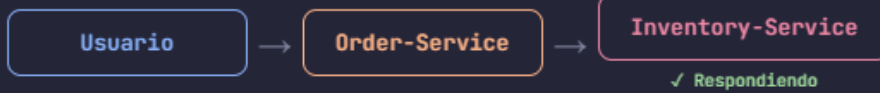
¿Qué pasa entonces?

- 1 **Order-Service** se queda **esperando**. El hilo que atendió esa petición está bloqueado.
- 2 Entran 1000 usuarios más. 1000 hilos nuevos. Todos **esperando**.
- 3 El servidor se queda sin recursos. **Order-Service** **cae**.

Fallo en Cascada

Un solo servicio roto puede tumbar toda la plataforma

ESTADO NORMAL · LAS PETICIONES FLUYEN



↓ Inventory-Service se cuelga

ESTADO DE FALLO · LA CASCADA SE PROPAGA



La solución tiene nombre

Un patrón de diseño que existe precisamente para este problema

PATRÓN DE DISEÑO

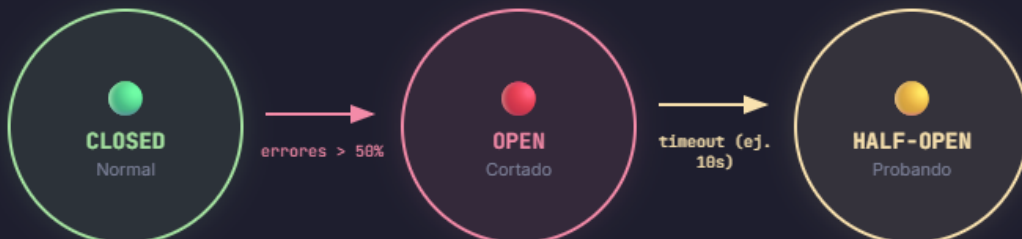
Circuit Breaker

Un mecanismo que **detecta fallos** repetidos en un servicio y **corta automáticamente** la comunicación antes de que el fallo se propague al resto del sistema.

El nombre viene de los sistemas eléctricos: cuando hay sobrecarga, el disyuntor corta el circuito para proteger la instalación. En software, aplicamos exactamente el mismo principio.

Los 3 estados del Circuit Breaker

El corazón del patrón: tres condiciones que se transicionen según los resultados



✓ Peticiones de prueba **pasan** → vuelve a **CLOSED**

✗ Peticiones de prueba **fallan** → vuelve a **OPEN**

¿Qué hace cada estado?

El comportamiento concreto en cada fase

CLOSED

Estado por defecto

Las peticiones pasan normalmente de **Order** a **Inventory**.

El sistema **cuenta los errores**. Cuando superan el umbral configurado, el circuito se abre.

OPEN

Circuito cortado

Ninguna petición llega a **Inventory**.

Si **Order** intenta llamarlo, el CB lo detiene **inmediatamente** y retorna un fallback sin esperar el timeout.

HALF-OPEN

Probando

Después del timeout, deja pasar algunas peticiones de prueba.

Si funcionan → sistema recuperado, vuelve a CLOSED.
Si fallan → sigue roto, vuelve a OPEN.

Mismo escenario, con Circuit Breaker

Exactamente la misma situación del slide 03, pero ahora con protección activa

CON CIRCUIT BREAKER · EL FALLO SE CONTIENE



Con Circuit Breaker

- 1 **Inventory** falla repetidamente → el CB abre el circuito.
- 2 **Order** no espera. Retorna un **fallback** inmediatamente.
- 3 Los hilos se liberan. El sistema sigue operando.

Sin Circuit Breaker

- 1 **Order** espera 30s por cada petición.
- 2 1000 hilos bloqueados simultáneamente.
- 3 El servidor colapsa. Todo cae.

Lo que implementaremos

En esta sección usaremos **Resilience4j** para controlar estos patrones

TIMEOUT

TimeLimiter

Define un límite de tiempo máximo para una operación. Si no responde dentro de ese límite, la corta. No seguimos esperando.

CIRCUIT BREAKER

CircuitBreaker

El patrón que vimos hoy. Detecta errores repetidos y corta automáticamente la comunicación para proteger el sistema.

RETRY

Retry

Si una petición falla por algo transitorio, la reintenta automáticamente antes de declarar error definitivo.



Resilience4j es la librería estándar de Spring Cloud para esto. En la clase 8.2 la instalamos y la configuramos directamente en **Order-Service**.

Contenido de la clase 8.2

- 1 Agregar **resilience4j** al proyecto
- 2 Configurar Circuit Breaker en **Order-Service**
- 3 Simular un fallo y ver la transición de estados
- 4 Implementar un fallback real

110. Setup de Resilience4j y seguridad Actuator

<https://gist.github.com/gchaldy/4f28a02ad445702927b1227d094a8540>

➔ Agregar dependencia en order-service:

<!-- Dependencias para circuit braker -->

<!-- Autoconfigurador (beans, circuit braker, etc), habilitar las anotaciones -->

<dependency>

<groupId>io.github.resilience4j</groupId>

<artifactId>resilience4j-spring-boot3</artifactId>

<version>2.3.0</version>

</dependency>

<!-- Guardián -->

<dependency>

<groupId>org.springframework.boot</groupId>

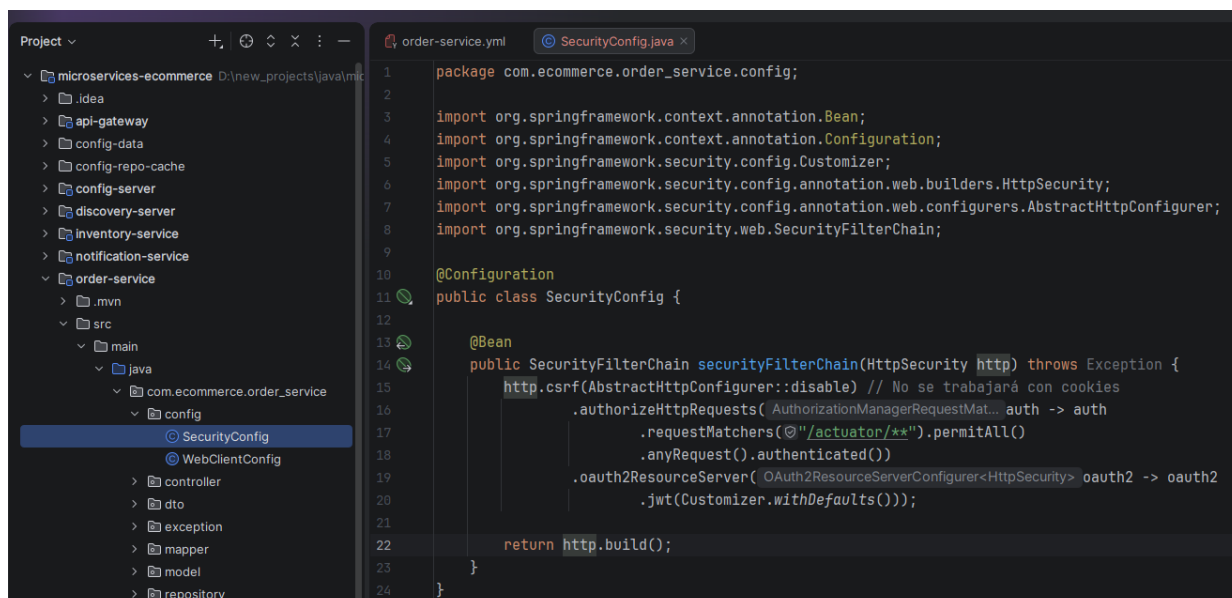
<artifactId>spring-boot-starter-aop</artifactId>

<version>3.3.0</version>

</dependency>

Nota: Antes resilience4j venía integrado en spring-cloud. En spring boot 4 no viene integrado.

➔ Dependencia oauth2-resource-server en order-service, por default tiene spring-security, esto hace que se tenga bloqueadas algunas rutas, es decir, si se quiere ingresar a “/actuator” y ver su configuración, no se podrá ingresar (está bloqueado por default). Para solventar ello, se realizará una clase de configuración donde solo se habilite a Actuator y permita la ruta sin la necesidad de un token.

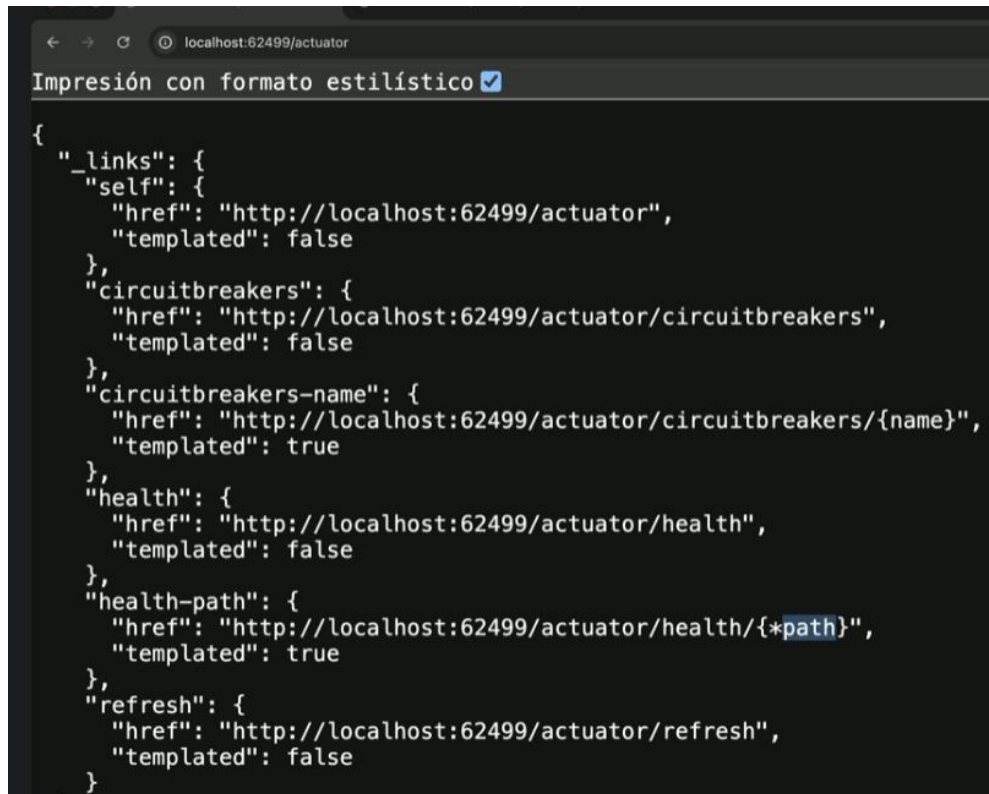


The screenshot shows an IDE with a project structure on the left and the `SecurityConfig.java` file open in the editor. The project structure includes a `microservices-e-commerce` project with several sub-projects, including `order-service`. The `SecurityConfig.java` file is located in the `com.ecommerce.order_service.config` package. It imports `org.springframework.context.annotation.Bean`, `org.springframework.context.annotation.Configuration`, `org.springframework.security.config.Customizer`, `org.springframework.security.config.annotation.web.builders.HttpSecurity`, `org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer`, and `org.springframework.security.web.SecurityFilterChain`. The class `SecurityConfig` is annotated with `@Configuration` and `@Bean`. It defines a `securityFilterChain` method that configures `HttpSecurity` to allow access to the `/actuator/**` path without authentication. The method returns `http.build()`.

```
1 package com.ecommerce.order_service.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.Customizer;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
8 import org.springframework.security.web.SecurityFilterChain;
9
10 @Configuration
11 public class SecurityConfig {
12
13     @Bean
14     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
15         http.csrf(AbstractHttpConfigurer::disable) // No se trabajará con cookies
16             .authorizeHttpRequests((AuthorizationManagerRequestMat... auth -> auth
17                 .requestMatchers(@"/actuator/**").permitAll()
18                 .anyRequest().authenticated())
19             .oauth2ResourceServer((OAuth2ResourceServerConfigurer<HttpSecurity> oauth2 -> oauth2
20                 .jwt(Customizer.withDefaults()));
21
22         return http.build();
23     }
24 }
```

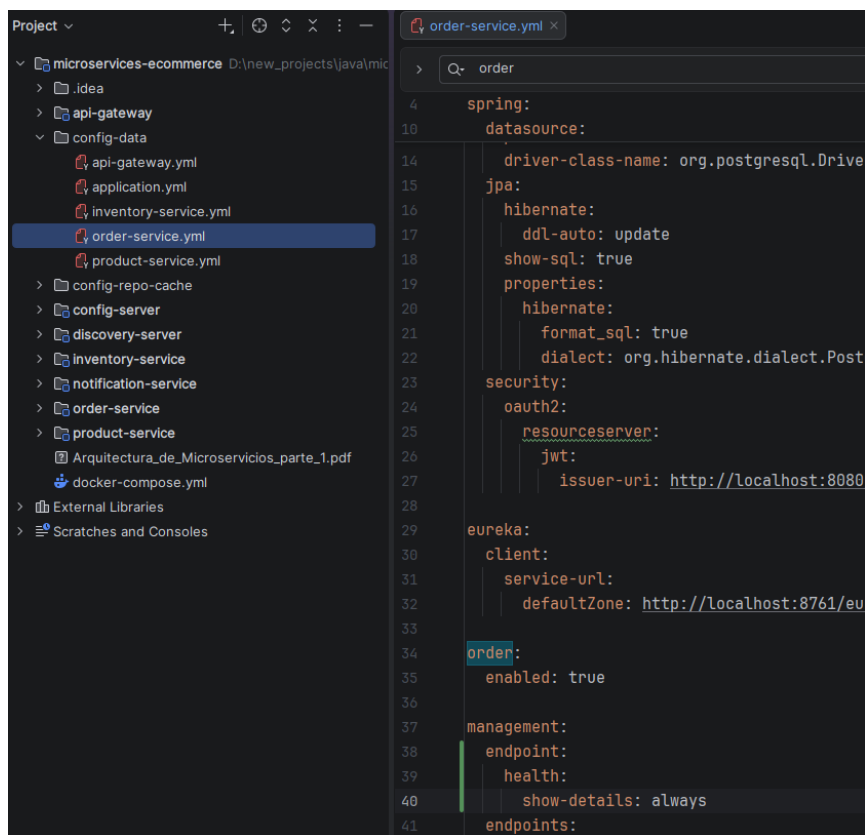
111. Introducción a Actuator

➔ Luego de la configuración de seguridad para path “/actuator” (ruta liberada):



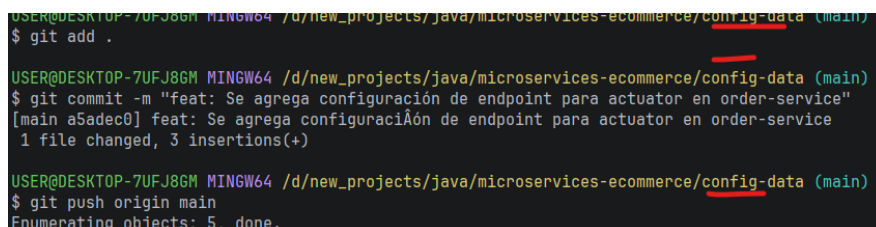
```
{
  "_links": {
    "self": {
      "href": "http://localhost:62499/actuator",
      "templated": false
    },
    "circuitbreakers": {
      "href": "http://localhost:62499/actuator/circuitbreakers",
      "templated": false
    },
    "circuitbreakers-name": {
      "href": "http://localhost:62499/actuator/circuitbreakers/{name}",
      "templated": true
    },
    "health": {
      "href": "http://localhost:62499/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:62499/actuator/health/{*path}",
      "templated": true
    },
    "refresh": {
      "href": "http://localhost:62499/actuator/refresh",
      "templated": false
    }
  }
}
```

➔ Para habilitar las demás rutas:



```
order-service.yml
4  spring:
10    datasource:
14      driver-class-name: org.postgresql.Driver
15
16    jpa:
17      hibernate:
18        ddl-auto: update
19        show-sql: true
20      properties:
21        hibernate:
22          format_sql: true
23          dialect: org.hibernate.dialect.PostgreSQLDialect
24
25    security:
26      oauth2:
27        resourceserver:
28          jwt:
29            issuer-uri: http://localhost:8080
30
31    eureka:
32      client:
33        service-url:
34          defaultZone: http://localhost:8761/eureka
35
36    order:
37      enabled: true
38
39    management:
40      endpoint:
41        health:
42          show-details: always
43      endpoints:
```

➔ Commitar



```
USER@DESKTOP-7UFG8GM MINGW64 /d/new_projects/java/microservices-ecommerce/config-data (main)
$ git add .

USER@DESKTOP-7UFG8GM MINGW64 /d/new_projects/java/microservices-ecommerce/config-data (main)
$ git commit -m "feat: Se agrega configuración de endpoint para actuator en order-service"
[main a5adec0] feat: Se agrega configuración de endpoint para actuator en order-service
1 file changed, 3 insertions(+)

USER@DESKTOP-7UFG8GM MINGW64 /d/new_projects/java/microservices-ecommerce/config-data (main)
$ git push origin main
Enumerating objects: 5, done.
```

→ Luego de liberar las rutas:

Impresión con formato estilístico ☒

```
{
  "components": {
    "clientConfigServer": {
      "details": {
        "propertySources": [
          "configserver:https://github.com/gchaldy/microservices-config-data.git/order-service.yml"
        ],
        "configClient": {}
      },
      "status": "UP"
    },
    "db": {
      "details": {
        "database": "PostgreSQL",
        "validationQuery": "isValid()"
      },
      "status": "UP"
    },
    "discoveryComposite": {
      "components": {
        "discoveryClient": {
          "details": {
            "services": [
              "api-gateway",
              "order-service"
            ]
          }
        }
      }
    }
  }
}
```

Spring Boot Actuator es una herramienta que añade funcionalidades de **monitorización y gestión a tus aplicaciones**. Permite observar el estado interno de tu aplicación (como uso de memoria, métricas y estado de salud) en tiempo real, facilitando la administración de microservicios, especialmente en entornos de producción.  Medium · Luis Cualquiera +2

¿Cómo funciona?

El Actuator expone una serie de *endpoints* (rutas web) o conexiones JMX para que puedas consultar información del sistema sin tener que programar estas funciones desde cero.  Medium · Luis Cualquiera +2

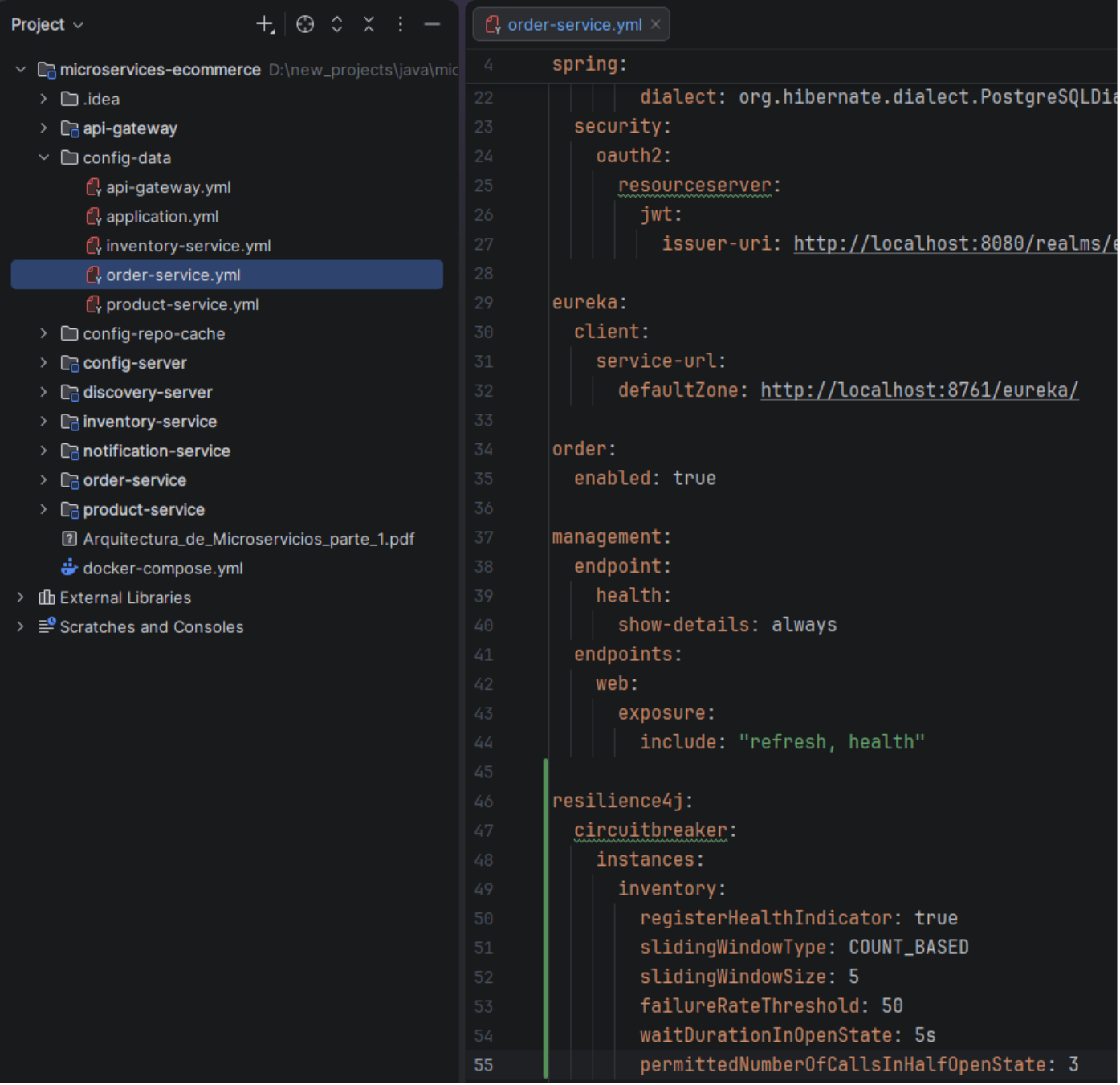
Para empezar a utilizarlo, solo necesitas añadir la dependencia `spring-boot-starter-actuator` en tu proyecto. Los endpoints más comunes que proporciona incluyen:  - Arquitectura Java +1

- `/actuator/health` : Devuelve el estado operativo de la aplicación (suele indicar si está UP O DOWN).
- `/actuator/metrics` : Muestra una amplia variedad de métricas del sistema, como el uso de memoria JVM, hilos (*threads*) y peticiones HTTP.
- `/actuator/beans` : Lista todos los beans de Spring que están cargados en el contexto de la aplicación.
- `/actuator/env` : Detalla las propiedades del entorno, variables de sistema y parámetros de configuración.  Medium · Luis Cualquiera +4

112. Circuit Breaker: Implementación

<https://gist.github.com/gchaldou/7543d4919b7fcddeee531cf836eb18f4>

→ En config-data: order-service, configuración



```
4      spring:
22          dialect: org.hibernate.dialect.PostgreSQLDialect
23      security:
24          oauth2:
25              resourceserver:
26                  jwt:
27                      issuer-uri: http://localhost:8080/realms/...
28
29      eureka:
30          client:
31              service-url:
32                  defaultZone: http://localhost:8761/eureka/
33
34      order:
35          enabled: true
36
37      management:
38          endpoint:
39              health:
40                  show-details: always
41          endpoints:
42              web:
43                  exposure:
44                      include: "refresh, health"
45
46      resilience4j:
47          circuitbreaker:
48              instances:
49                  inventory:
50                      registerHealthIndicator: true
51                      slidingWindowType: COUNT_BASED
52                      slidingWindowSize: 5
53                      failureRateThreshold: 50
54                      waitDurationInOpenState: 5s
55                      permittedNumberOfCallsInHalfOpenState: 3
```

→ Commitear

Inventory → Nombre del corto circuito, debe coincidir con el nombre de la anotación de java (por hacer)

registerHealthIndicator: true → Activa la vigilancia del Actuator

slidingWindowType: COUNT_BASED → Definine cómo mide los fallos (en este caso que cuente la cantidad de peticiones; también se puede configurar por tiempo)

slidingWindowSize: 5 → Tamaño de la memoria, últimas 5 peticiones para tomar decisiones.

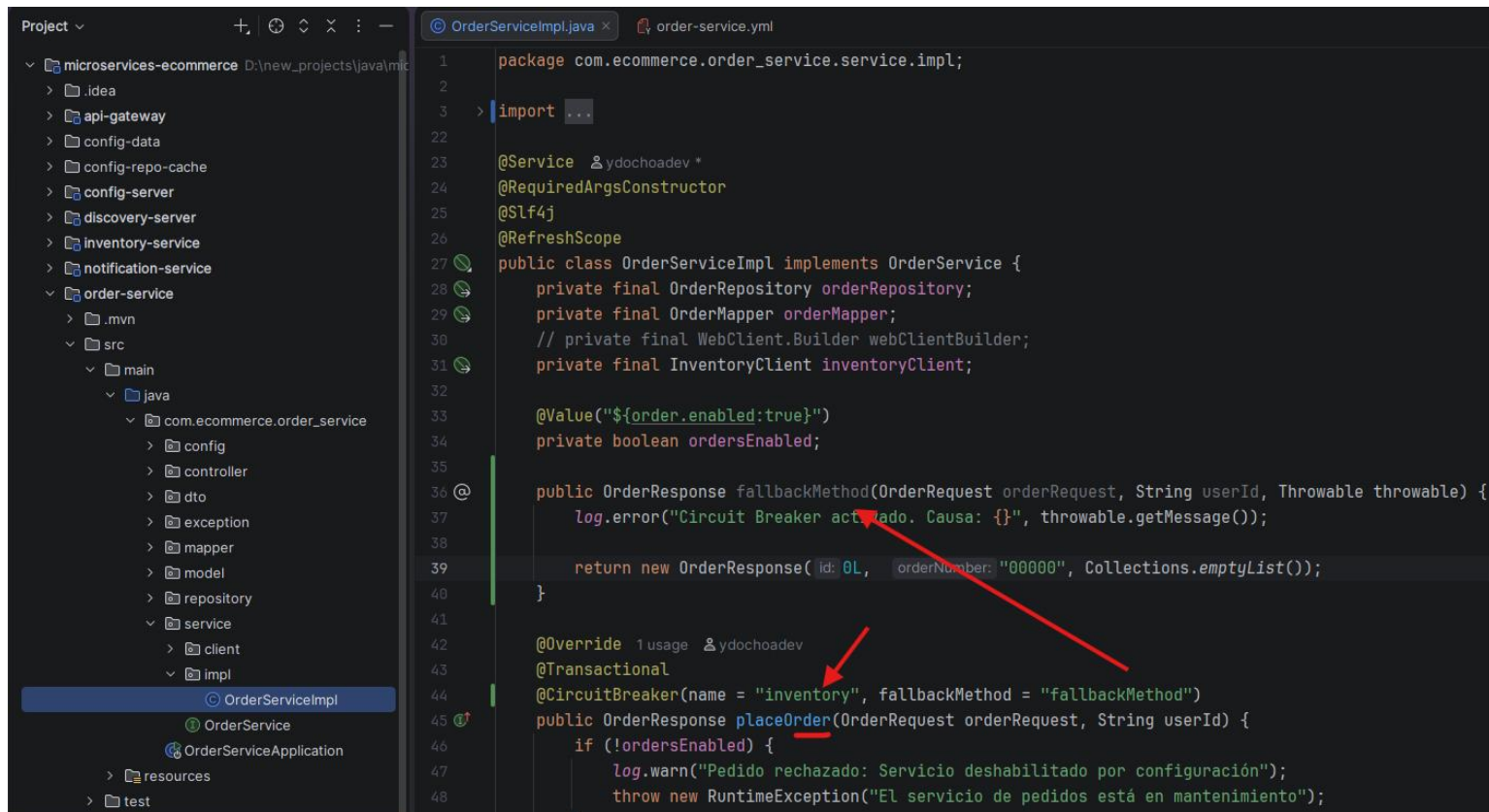
failureRateThreshold: 50 → Tasa de tolerancia a fallos (50% de esas 5 peticiones (si fallan 3 de esas 5, corta el circuito, se abre)

waitDurationInOpenState: 5s → Una vez abierto el circuito, espera 5 segundos, rechazando todo antes de intentar reconectar

permittedNumberOfCallsInHalfOpenState: 3 → Prueba de confianza, pasados los 5 segundos, va a pasar solo algunas peticiones (prueba). Deja pasar 3 peticiones. Si estas 3 funcionan, cierra el circuito y vuelve el sistema a la normalidad.

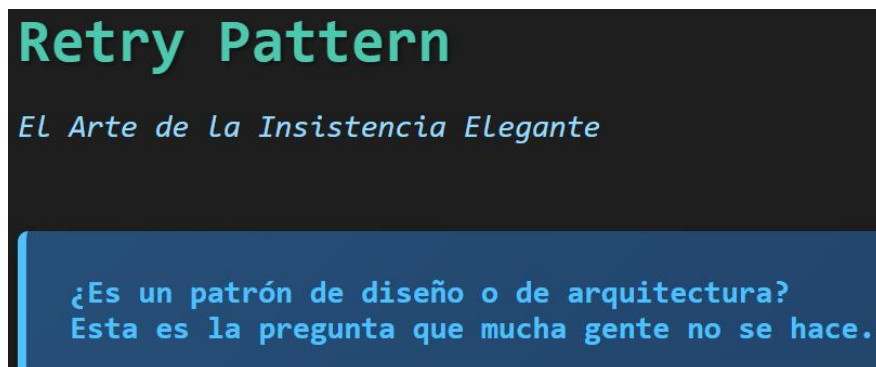
→ Clase de implementación:

Al realizar el pedido. Hacer referencia a la configuración



```
1 package com.ecommerce.order_service.service.impl;
2
3 import ...
4
22
23 @Service
24 @RequiredArgsConstructor
25 @Slf4j
26 @RefreshScope
27 public class OrderServiceImpl implements OrderService {
28     private final OrderRepository orderRepository;
29     private final OrderMapper orderMapper;
30     // private final WebClient.Builder webClientBuilder;
31     private final InventoryClient inventoryClient;
32
33     @Value("${order.enabled:true}")
34     private boolean ordersEnabled;
35
36     @Override
37     public OrderResponse fallbackMethod(OrderRequest orderRequest, String userId, Throwable throwable) {
38         log.error("Circuit Breaker activado. Causa: {}", throwable.getMessage());
39         return new OrderResponse(id: 0L, orderNumber: "00000", Collections.emptyList());
40     }
41
42     @Override
43     @Transactional
44     @CircuitBreaker(name = "inventory", fallbackMethod = "fallbackMethod")
45     public OrderResponse placeOrder(OrderRequest orderRequest, String userId) {
46         if (!ordersEnabled) {
47             log.warn("Pedido rechazado: Servicio deshabilitado por configuración");
48             throw new RuntimeException("El servicio de pedidos está en mantenimiento");
49         }
50     }
51 }
```

113. Retry pattern ¿Qué es?



Retry Pattern

El Arte de La Insistencia Elegante

¿Es un patrón de diseño o de arquitectura?
Esta es la pregunta que mucha gente no se hace.

¿Qué es realmente el Retry Pattern?



Clasificación Técnica

El Retry Pattern se clasifica como un **Patrón de Estabilidad** o **Cloud Design Pattern** (Patrón de Diseño en la Nube).

Aunque se implementa en el código (diseño), su impacto y propósito son **arquitectónicos**.

¿Por qué es un híbrido?

Como Patrón de Diseño: Se parece a un Decorator o Proxy. Envuelves tu llamada original en una lógica que la repite.

Como Patrón Arquitectónico: Es vital para la Resiliencia. Define cómo se comportan tus servicios cuando la red es hostil.

El Concepto Fundamental

Es una estrategia de manejo de errores que asume que ciertas fallas son temporales (transitorias).

"Si falló ahora, quizás en 200 milisegundos funcione"

⚡ La premisa del Retry: No te rindas a la primera. Espera un poco y prueba de nuevo.

¿Por qué es más relevante en Microservicios?



Monolito

Todo está en una sola memoria.

Si algo falla, suele ser definitivo:

- NullPointerException
- Error de lógica
- Datos inválidos
- División por cero

✗ Sin red = Sin errores transitorios

Los fallos son deterministas y reproducibles.



Microservicios

Todo está distribuido en red.

El 90% de los errores son de red:

- Un parpadeo en el WiFi
- Balanceador cambiando instancias
- Base de datos bloqueada 5ms
- Timeout por congestión

✓ La red es caótica = Retry es esencial

La mayoría de estos errores desaparecen en milisegundos.

Sin Retry, una arquitectura de microservicios es extremadamente frágil. Un solo fallo en cadena rompe todo.

| Las 3 Reglas de Oro

Un buen Retry no es solo un `while(true)`

1

Count (Cantidad)

¿Cuántas veces insisto?

Ejemplo: `3 veces`

Si insistes infinitamente, bloqueas tu propio sistema.

2

Interval (Espera)

¿Cuánto espero entre intentos?

Ejemplo: `2 segundos`

Dar tiempo al servicio para recuperarse.

3

Backoff (Retroceso)

Esta es la "inteligencia" del patrón.

Define cómo crece el tiempo de espera entre intentos.

| Estrategias de Backoff

La inteligencia del Retry



Fixed Backoff (Fijo)

Espero siempre el mismo tiempo: `2s → 2s → 2s`

Problema: Si el servicio está saturado, seguirás bombardeándolo con la misma frecuencia.



Exponential Backoff (Exponencial) ✨

El tiempo de espera crece exponencialmente: `1s → 2s → 4s → 8s`



Best Practice: No saturas al servicio que ya está sufriendo. Le das cada vez más tiempo para recuperarse.



Exponential Backoff es la estrategia recomendada por AWS, Google Cloud y Azure para sistemas distribuidos.

| ¿Cuándo NO usarlo?

La trampa del Retry mal aplicado

El Retry solo sirve para Errores Transitorios

✅ Úsalo para:

- 🚧 503 Service Unavailable
- 🕒 Timeout
- 🚫 Connection Refused
- 🌐 Network Error
- 📶 429 Too Many Requests

Por qué: Estos errores son temporales. El servicio puede estar reiniciándose, el balanceador rotando instancias, o la red congestionada.

❌ NO lo uses para:

- 🔒 401 Unauthorized
- 📄 400 Bad Request
- 🔍 404 Not Found
- 🚫 403 Forbidden
- 💥 500 Internal Server Error

Por qué: Si te equivocaste en la contraseña, por más que reintentas 100 veces, seguirá estando mal. Solo gastarás recursos.

| Tipos de Errores en Detalle

Saber distinguirlos es crítico

✅ 503 Service Unavailable

El servicio está temporalmente fuera de línea. Reintentar en 2-5 segundos tiene sentido.

✅ Timeout

La red estaba lenta o el servidor sobrecargado. Puede funcionar en el siguiente intento.

✅ Connection Refused

El servicio está reiniciándose. En unos segundos podría estar disponible.

❌ 401 Unauthorized

Token inválido o expirado. Reintentar no cambiará el resultado. Necesitas un nuevo token.

❌ 400 Bad Request

Los datos están mal formados. El mismo request siempre dará el mismo error.

❌ 404 Not Found

El recurso no existe. Insistir 10 veces no lo creará mágicamente.

Ejemplo Práctico

Escenario Real

Tu sistema hace una petición HTTP a un microservicio de inventario. La red tiene un "parpadeo" de 0.5 segundos y la conexión falla.

❌ Sin Retry

Intento 1: Timeout
Resultado: Error al usuario
Usuario: 😡 "La app no funciona"

Pérdida de negocio

El usuario ve un error por un problema de red de medio segundo. Mala experiencia.

✅ Con Retry (Exponential Backoff)

Intento 1: Timeout (1s)
Espera: 1 segundo
Intento 2: ✅ Éxito
Usuario: 😊 "Todo funcionó"

Resiliencia automática

El usuario ni siquiera sabe que hubo un problema. Sistema robusto.

Entonces:

🎯 **Naturaleza:** Patrón de Estabilidad / Cloud Design Pattern

💡 **Propósito:** Manejar errores transitorios en sistemas distribuidos

⚙️ **Componentes:** Count (cantidad) + Interval (espera) + Backoff (estrategia)

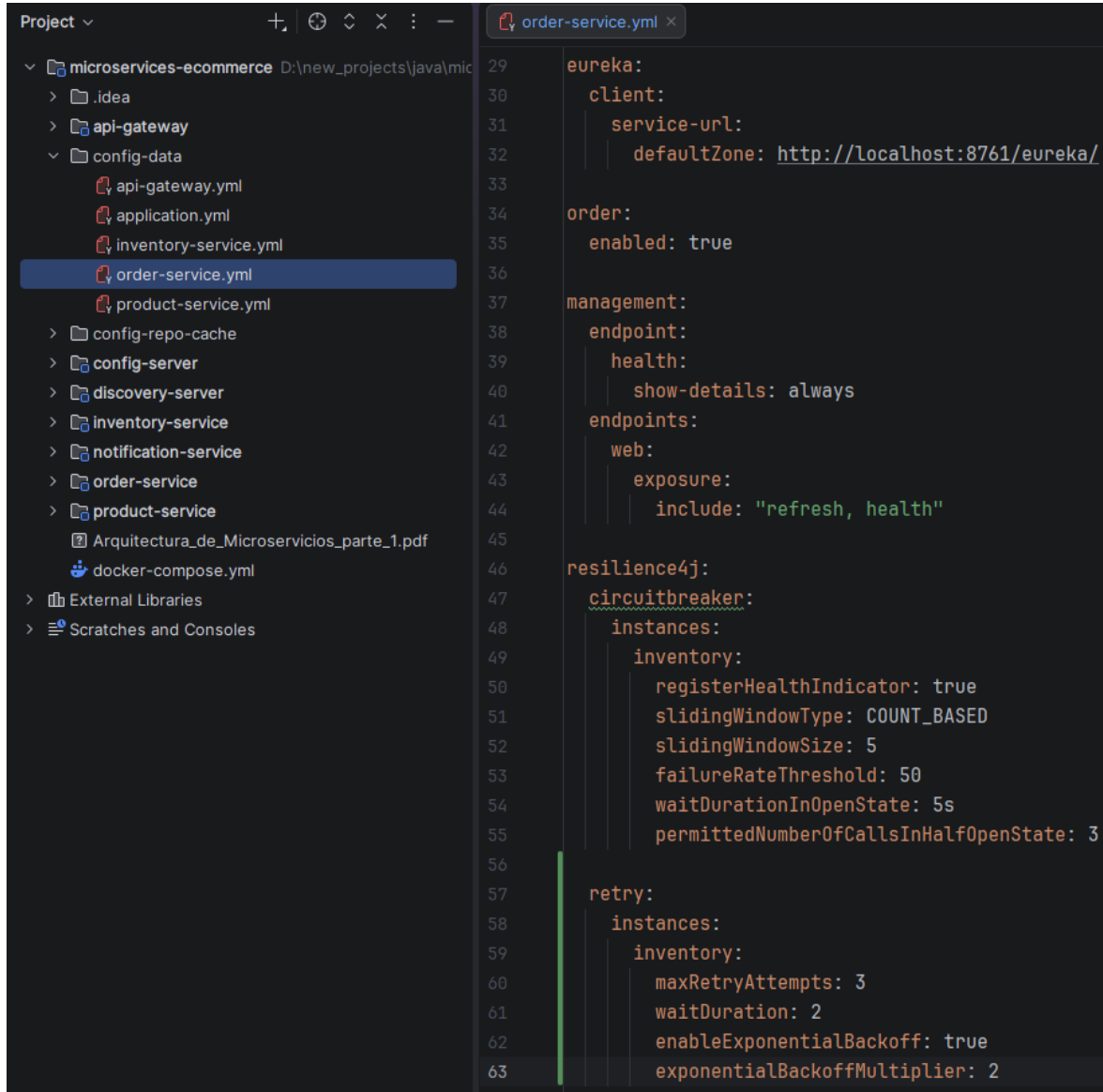
🚀 **Best Practice:** Exponential Backoff

⚠️ **Cuidado:** Solo para errores transitorios, no errores de lógica

"El Retry Pattern es el arte de la insistencia elegante.
Es la herramienta que evita que tu usuario vea una pantalla de error
solo porque un cable de red parpadeó durante 0.5 segundos."

114. Retry pattern: En el código

→ En config-data, oser-service.yml:



```
29 eureka:
30   client:
31     service-url:
32       defaultZone: http://localhost:8761/eureka/
33
34   order:
35     enabled: true
36
37   management:
38     endpoint:
39       health:
40         show-details: always
41     endpoints:
42       web:
43         exposure:
44           include: "refresh, health"
45
46   resilience4j:
47     circuitbreaker:
48       instances:
49         inventory:
50           registerHealthIndicator: true
51           slidingWindowType: COUNT_BASED
52           slidingWindowSize: 5
53           failureRateThreshold: 50
54           waitDurationInOpenState: 5s
55           permittedNumberOfCallsInHalfOpenState: 3
56
57   retry:
58     instances:
59       inventory:
60         maxRetryAttempts: 3
61         waitDuration: 2
62         enableExponentialBackoff: true
63         exponentialBackoffMultiplier: 2
```

maxRetryAttempts: 3 → Máximo 3 intentos

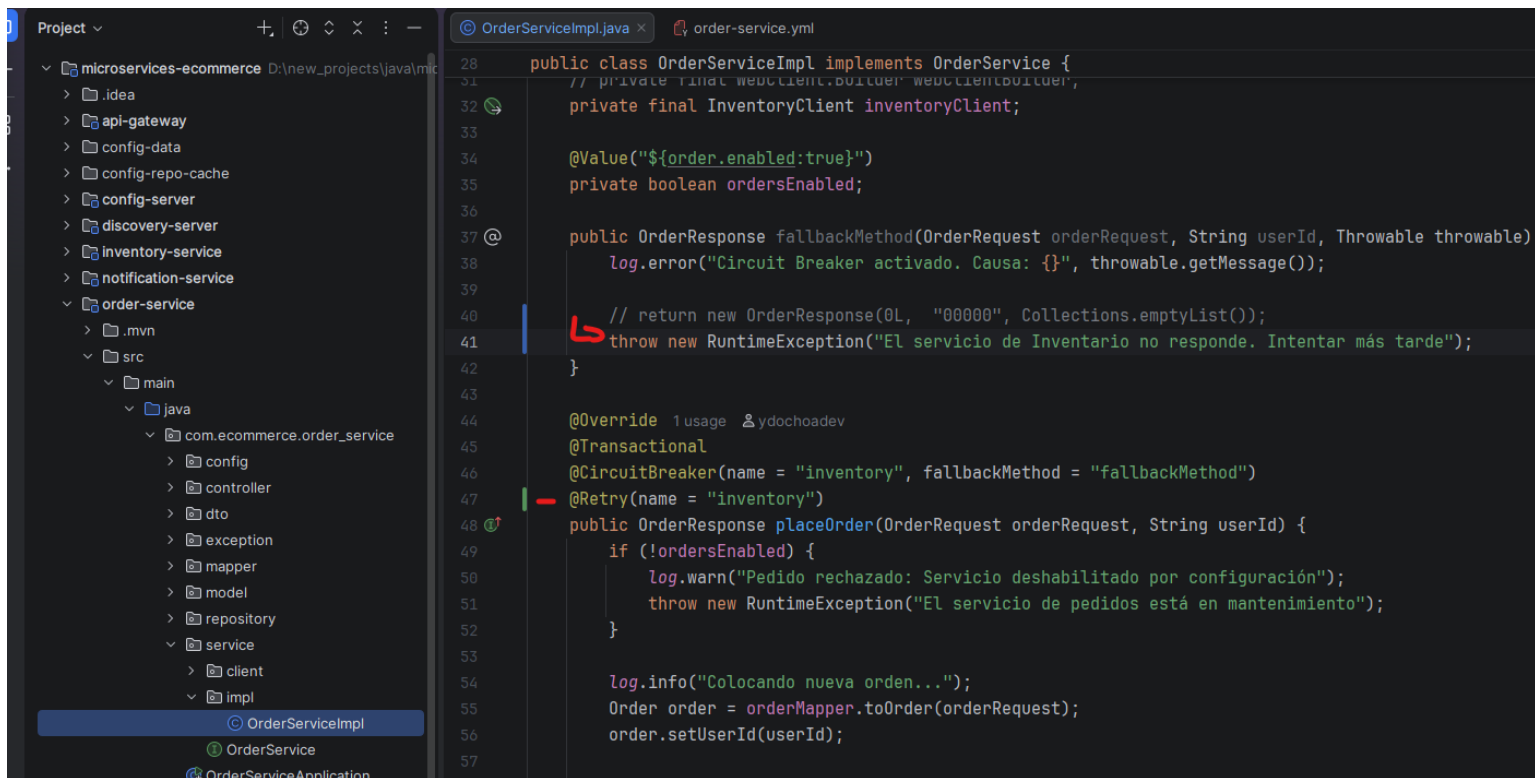
waitDuration: 2 → Intervalo inicial, esperamos 2 segundos

enableExponentialBackoff: true → Activamos la inteligencia, el exponencial

exponentialBackoffMultiplier: 2 → Multiplicador (waitDuration * exponentialBackoffMultiplier)

→ Commitear

→ En orderServiceImpl:



```
28 public class OrderServiceImpl implements OrderService {
29     // private final WebClient.Builder webClientBuilder;
30
31     private final InventoryClient inventoryClient;
32
33
34     @Value("${order.enabled:true}")
35     private boolean ordersEnabled;
36
37     @
38     public OrderResponse fallbackMethod(OrderRequest orderRequest, String userId, Throwable throwable) {
39         log.error("Circuit Breaker activado. Causa: {}", throwable.getMessage());
40
41         // return new OrderResponse(0L, "00000", Collections.emptyList());
42         throw new RuntimeException("El servicio de Inventario no responde. Intentar más tarde");
43     }
44
45     @Override @usage @ydochoadev
46     @Transactional
47     @CircuitBreaker(name = "inventory", fallbackMethod = "fallbackMethod")
48     @Retry(name = "inventory")
49     public OrderResponse placeOrder(OrderRequest orderRequest, String userId) {
50         if (!ordersEnabled) {
51             log.warn("Pedido rechazado: Servicio deshabilitado por configuración");
52             throw new RuntimeException("El servicio de pedidos está en mantenimiento");
53         }
54
55         log.info("Colocando nueva orden...");
56         Order order = orderMapper.toOrder(orderRequest);
57         order.setUserId(userId);
```

** Terminación abrupta en el método “fallbackMethod”: En java, una función debe tener una salida garantizada. Hay 2 tipos de salida: retorna el valor prometido (OrderResponse) y la 2º es abrupta, lanzamiento de excepción (como es imposible ejecutar cualquier línea posterior a la excepción, el compilador no exige un return final)

** El método “fallbackMethod” debe tener la misma estructura del método “placeOrder”.

116. Timeout pattern ¿Qué es?

Timeout Pattern

El Arte de Soltar (y fallar rápido)

En sistemas distribuidos, una respuesta lenta es mucho más peligrosa que una respuesta fallida.

El Asesino Silencioso: Latencia

Respuesta Lenta

El servicio tarda 30 segundos en responder.

El cliente se queda esperando (hilo bloqueado).

Resultado: Consumo de recursos.

Fallo Rápido

El servicio falla en 10ms (Excepción).

El cliente se libera inmediatamente.

Resultado: Sistema disponible.

¿Por qué explota el servidor?

Thread Pool Exhaustion (Agotamiento de Hilos)

Imagina que tu servidor tiene capacidad para atender 10 clientes simultáneos.



Si 10 usuarios piden algo lento, el usuario #11 es rechazado aunque el sistema no tenga errores.

La Solución: Poner un Cronómetro

El Timeout Pattern establece un límite máximo de espera.
"Si no terminas en 3 segundos, me voy."

⚙️ Cómo funciona

1. Iniciamos la petición y arrancamos un timer.
2. Si la respuesta llega antes del timer ✅ → Todo bien.
3. Si el timer suena antes ❌ → Cancelamos la ejecución y lanzamos `TimeoutException`.

La Trinidad de la Resiliencia

🛡️ Circuit Breaker

"Evita llamar si ya sabemos que está roto."

🔄 Retry

"Intenta de nuevo si fue un parpadeo."

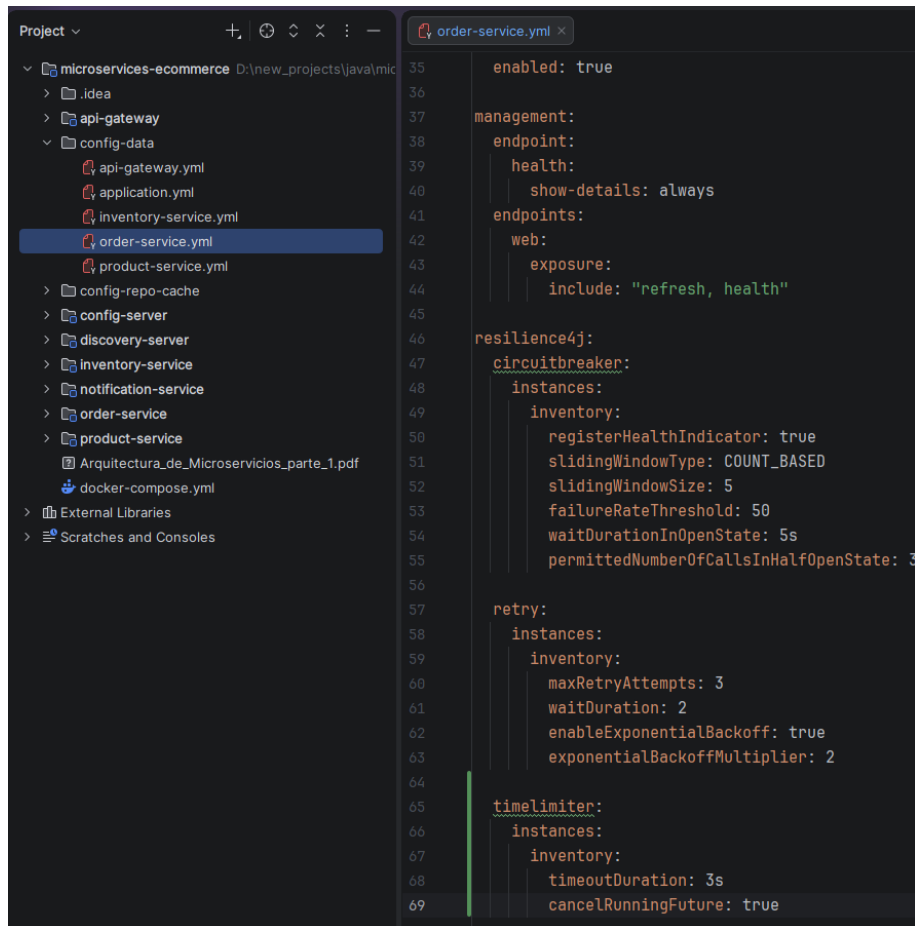
⌚ Timeout

"No esperes por siempre."

Resumen: Esperar consume más recursos que fallar. Un buen arquitecto prefiere un error 503 rápido que un servidor colgado eternamente.

117. Timeouts: Configuración

➔ En order-service, en config-data:



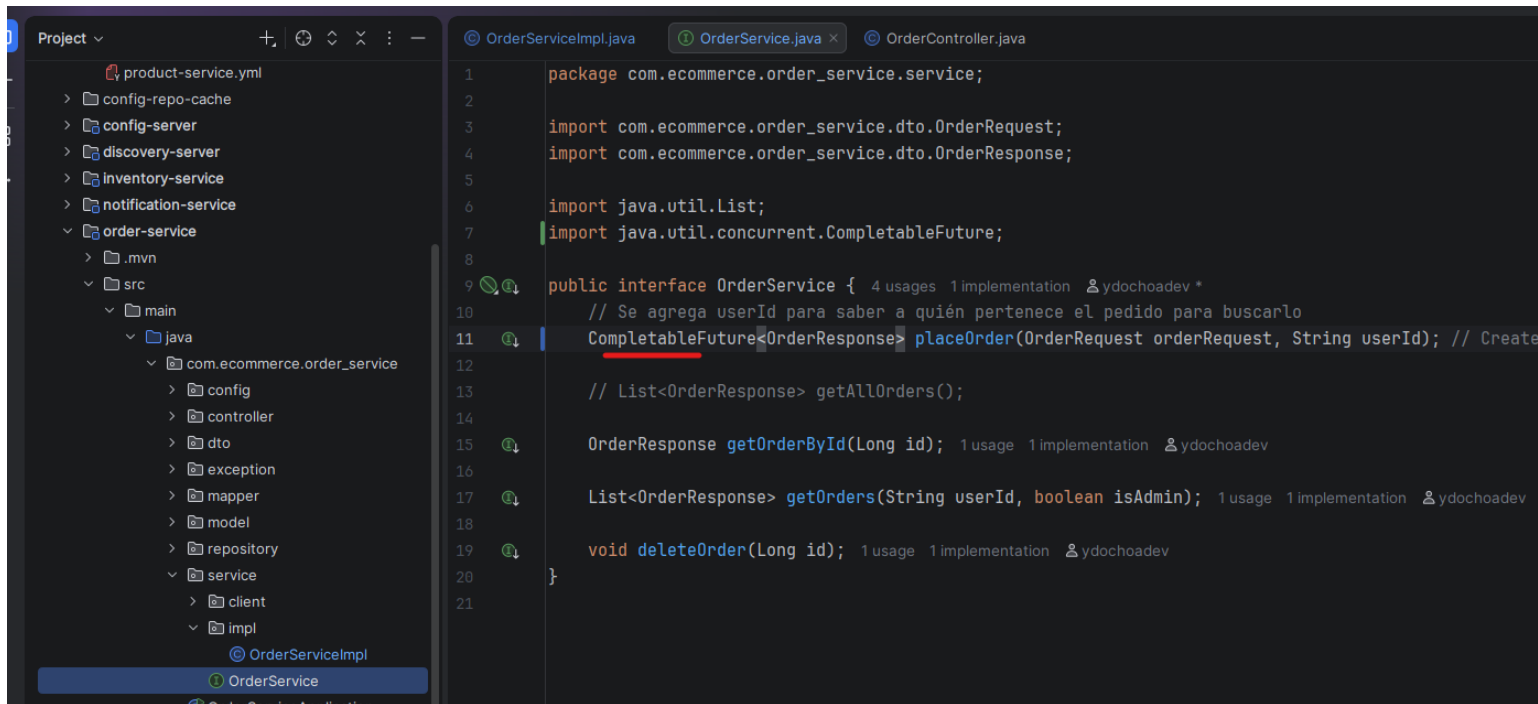
timeoutDuration: 3s ➔ 3 segundos máximo de espera.

cancelRunningFuture: true ➔ Pasados esos 3 segundos, matamos el hilo para liberar el recurso

➔ Commitear

➔ Aplicar time limiter: Para usar un time limiter con anotaciones, resilience4j pide un requisito: el método debe ser asíncorno. Para poder cortar la llamada a los 3 segundos, necesita que se ejcute en un hilo separado.

➔ En order-service:



➔ Implementación

@Override

@Transactional

@CircuitBreaker(name = "inventory", fallbackMethod = "fallbackMethod")

@Retry(name = "inventory")

@TimeLimiter(name = "inventory")

```
public CompletableFuture<OrderResponse> placeOrder(OrderRequest orderRequest, String userId) {  
    long startTime = System.currentTimeMillis();
```

```
    return CompletableFuture.supplyAsync() -> {  
        if (!ordersEnabled) {  
            log.warn("Pedido rechazado: Servicio deshabilitado por configuración");  
            throw new RuntimeException("El servicio de pedidos está en mantenimiento");  
        }  
    }
```

```
    log.info("Colocando nueva orden...");  
    Order order = orderMapper.toOrder(orderRequest);  
    order.setUserId(userId);
```

```
    for (var item : order.getOrderLineItemsList()) {  
        String sku = item.getSku();  
        Integer quantity = item.getQuantity();  
        // Realizar la llamada a inventory  
        try {  
            inventoryClient.reduceStock(sku, quantity);  
        } catch (Exception e) {  
            log.error("Error al reducir stock del producto {}. {}", sku, e.getMessage());  
            throw new IllegalArgumentException("No se pudo procesa la orden: Stock insuficiente o error de inventario");  
        }  
    }
```

```

    }

    order.setOrderNumber(UUID.randomUUID().toString());

    long totalTime = System.currentTimeMillis() - startTime;
    if (totalTime > 3000) {
        log.warn("Timeout detectado internamente ({} ms). Abortando guardado en BD", totalTime);
        throw new RuntimeException("Timeout excedido - Rollback manual");
    }
    // Guardamos y capturamos la entidad persistida
    Order savedOrder = orderRepository.save(order);
    log.info("Orden guardada con éxito. ID: {}", savedOrder.getId());

    return orderMapper.toOrderResponse(savedOrder);
});
}

```



118. El problema de la transacción distribuida

Cuando se realiza una orden y el inventory se tarda en responder, lanza excepción antes de guardar en BD la trx de la orden:

```

long totalTime = System.currentTimeMillis() - startTime;
if (totalTime > 3000) {
    log.warn("Timeout detectado internamente ({} ms). Abortando guardado en BD", totalTime);
    throw new RuntimeException("Timeout excedido - Rollback manual");
}
// Guardamos y capturamos la entidad persistida
Order savedOrder = orderRepository.save(order);
log.info("Orden guardada con éxito. ID: {}", savedOrder.getId());

```

Pero el servicio de inventory, reduce el stock a pesar de que la trx lanzó la excepción. Esto es un error: resilience4j reintentó conectarse al servicio 3 veces, el stock se descontó 3 veces.

```
for (var item : order.getOrderLineItemsList()) {
    String sku = item.getSku();
    Integer quantity = item.getQuantity();
    // Realizar la llamada a inventory
    try {
        inventoryClient.reduceStock(sku, quantity);
    } catch (Exception e) {
        log.error("Error al reducir stock del producto {}. {}", sku, e.getMessage());
        throw new IllegalArgumentException("No se pudo procesa la orden: Stock insuficiente o error de inventario")
    }
}

order.setOrderNumber(UUID.randomUUID().toString());

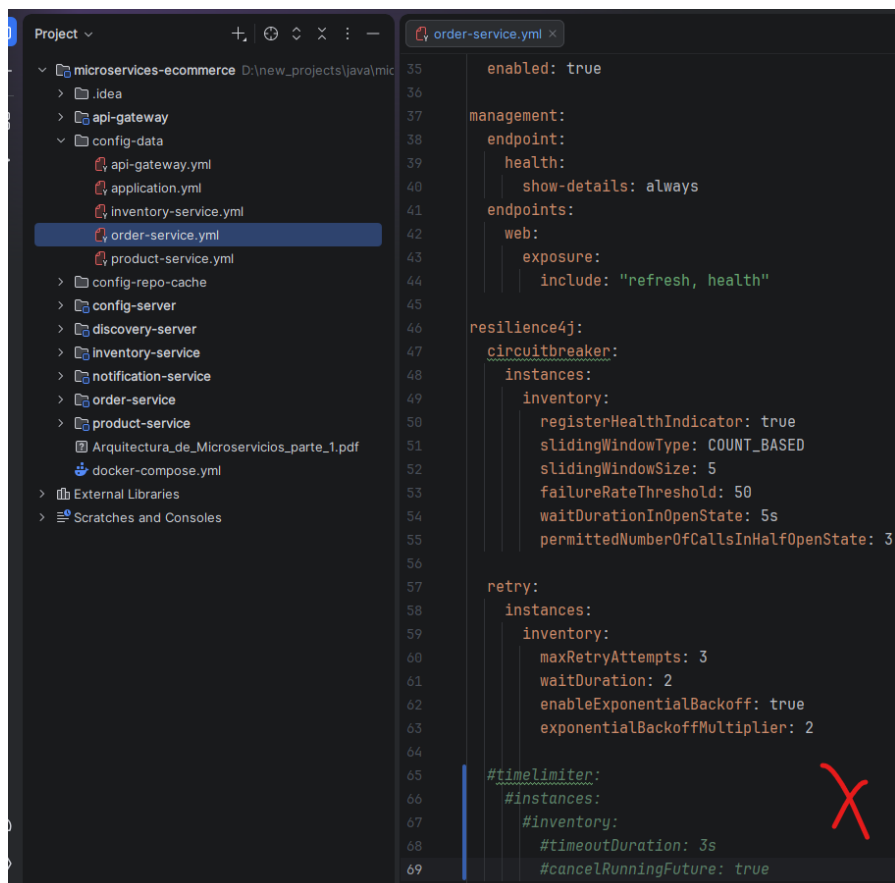
long totalTime = System.currentTimeMillis() - startTime;
if (totalTime > 3000) {
    log.warn("Timeout detectado internamente ({} ms). Abortando guardado en BD", totalTime);
    throw new RuntimeException("Timeout excedido - Rollback manual");
}
```

¿Qué pasó? Intentamos realizar una trx distribuida, el micro se order inició su trx local, al saltar el timeout hizo rollback y no se guardó nada. Pero el micro inventory inició su propia trx local, y nadie le avisó que cancelara, termina su trabajo e hizo su commit.

Para solventar esto, se necesita cambiar la arquitectura: dejar de usar llamadas de tipo Rest de forma directa; se debe pasar a una arquitectura orientada a eventos.

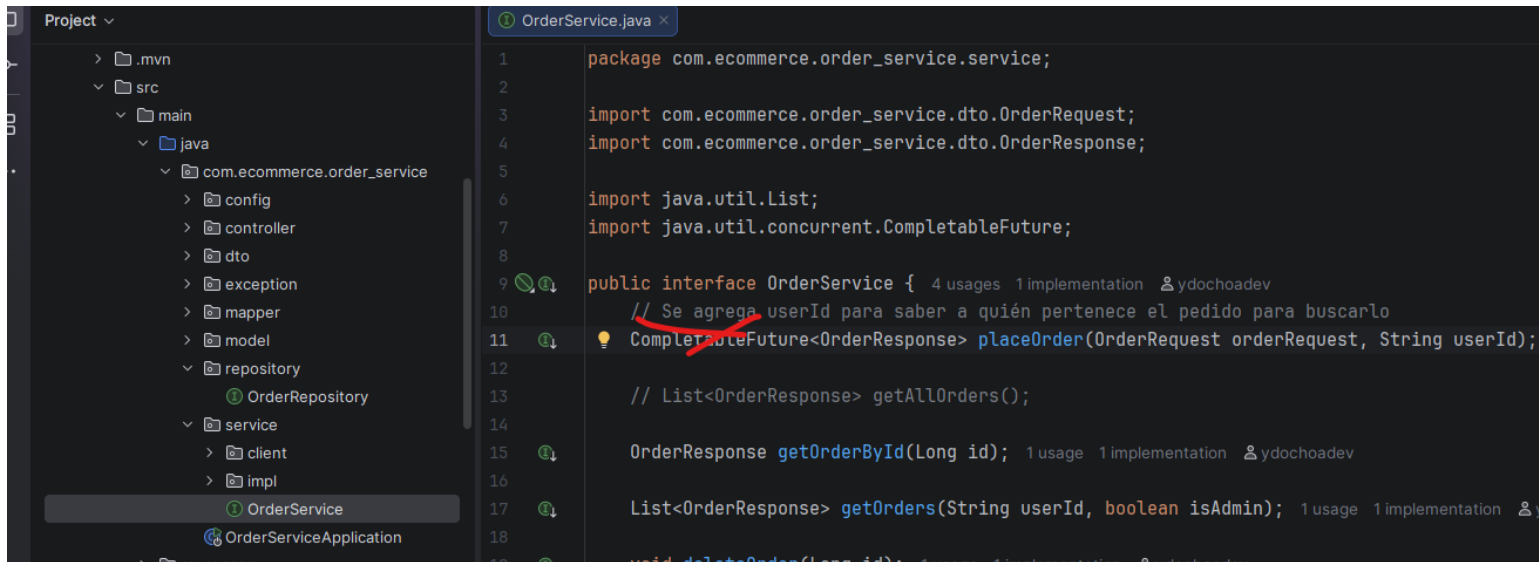
119. Preparación para eventos

➔ Limpiar la configuración de order-service.yml de config-data (quitando timelimiter): circuitBreaker y retry se quedan:



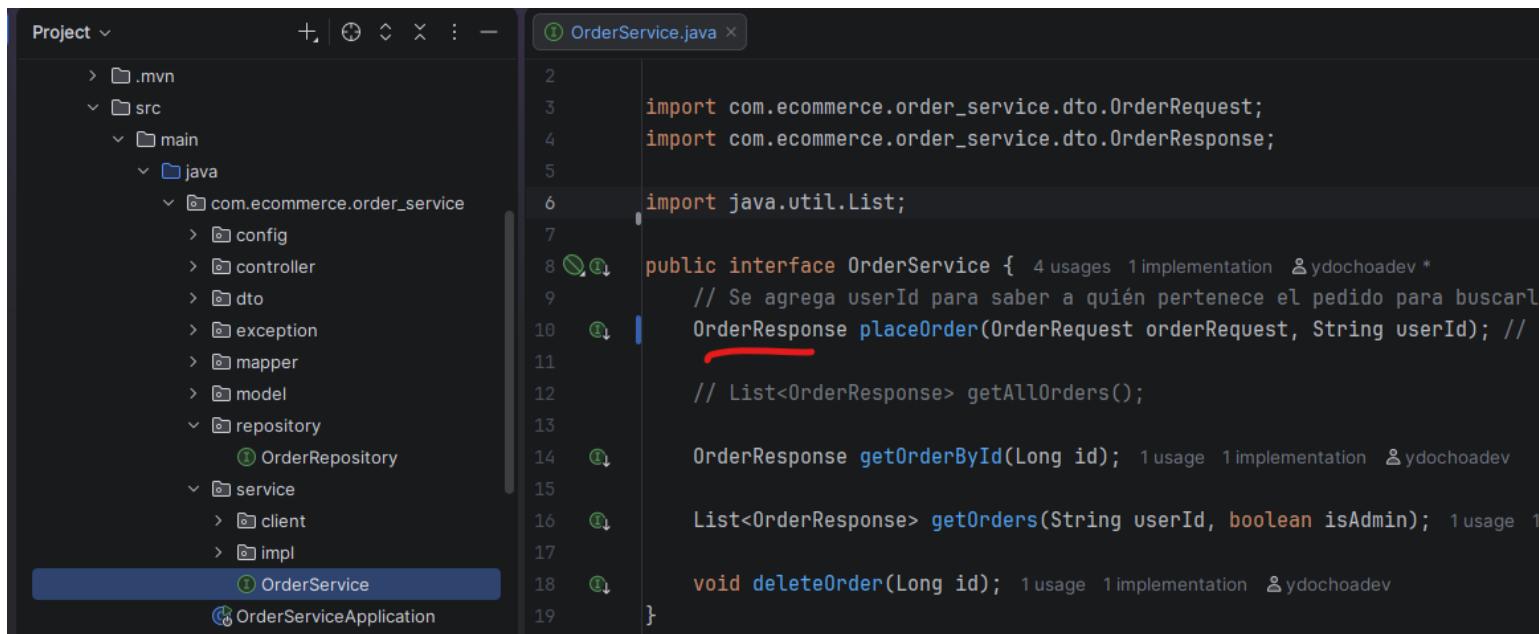
→ commitear

→ En la implementación:

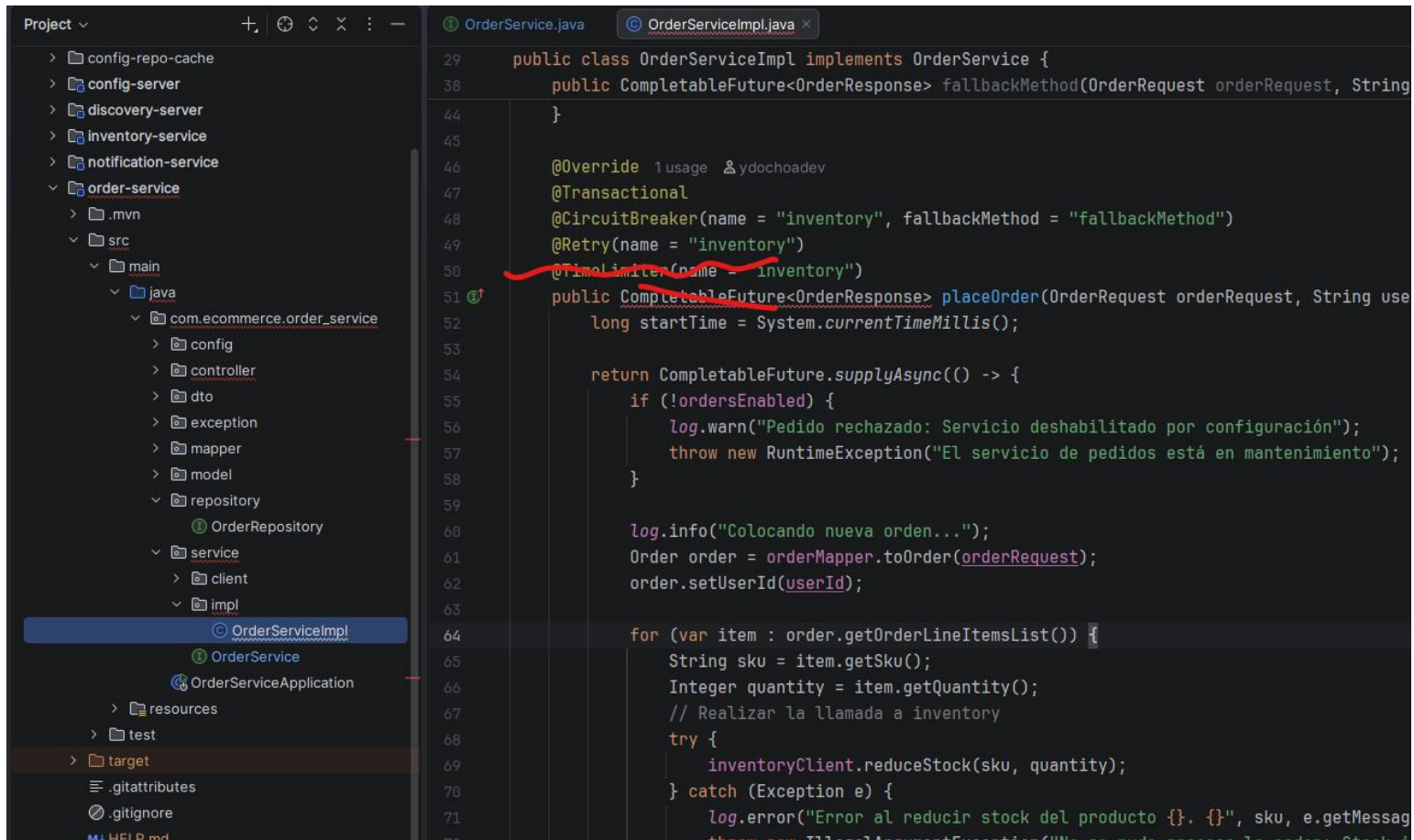


```
1 package com.ecommerce.order_service.service;
2
3 import com.ecommerce.order_service.dto.OrderRequest;
4 import com.ecommerce.order_service.dto.OrderResponse;
5
6 import java.util.List;
7 import java.util.concurrent.CompletableFuture;
8
9 public interface OrderService { 4 usages 1 implementation ydochoadev
10     // Se agrega userId para saber a quién pertenece el pedido para buscarlo
11     CompletableFuture<OrderResponse> placeOrder(OrderRequest orderRequest, String userId);
12
13     // List<OrderResponse> getAllOrders();
14
15     OrderResponse getOrderById(Long id); 1 usage 1 implementation ydochoadev
16
17     List<OrderResponse> getOrders(String userId, boolean isAdmin); 1 usage 1 implementation ydochoadev
18
19     void deleteOrder(Long id); 1 usage 1 implementation ydochoadev
```

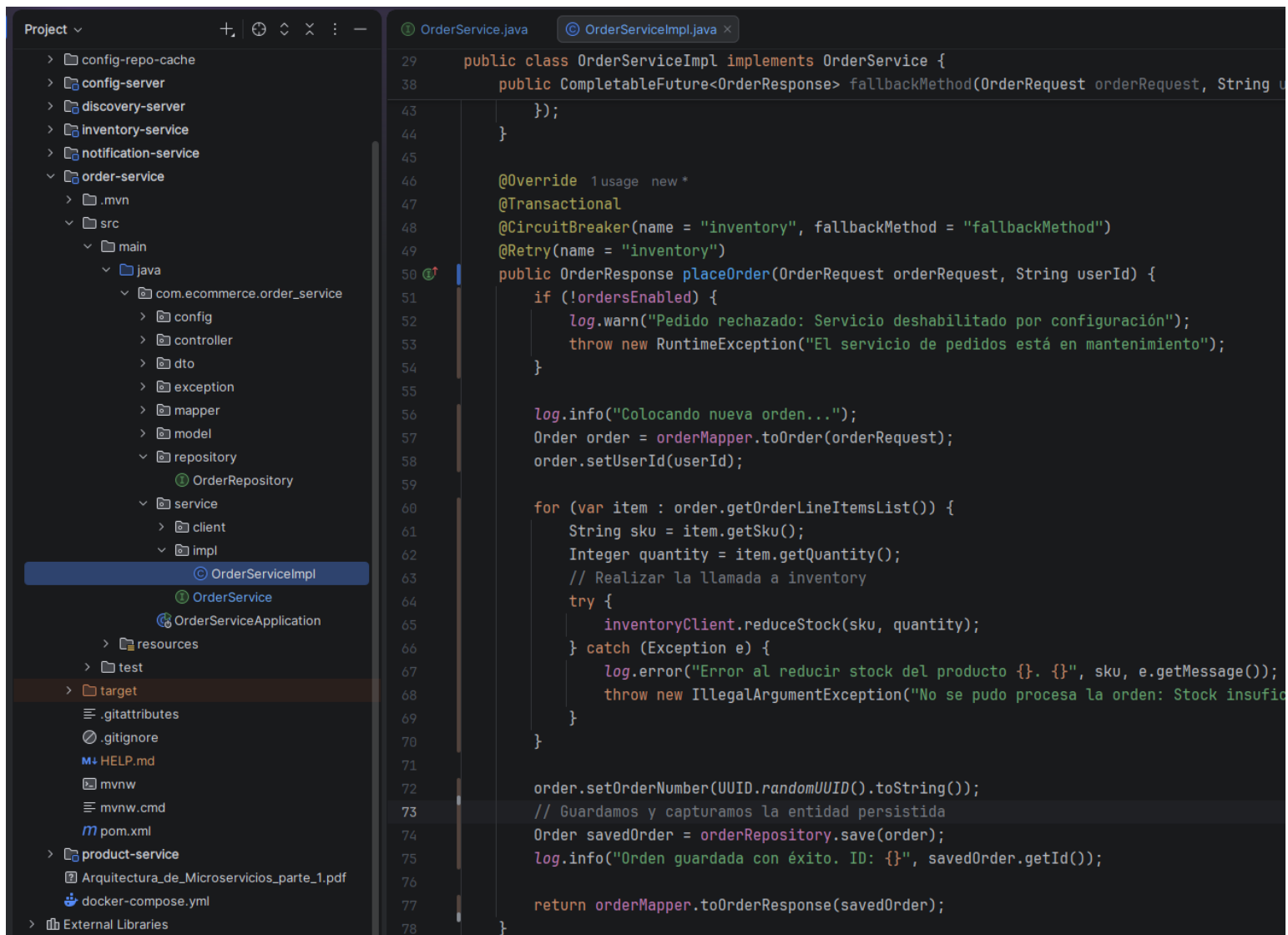
Quedaría:



```
2
3 import com.ecommerce.order_service.dto.OrderRequest;
4 import com.ecommerce.order_service.dto.OrderResponse;
5
6 import java.util.List;
7
8 public interface OrderService { 4 usages 1 implementation ydochoadev *
9     // Se agrega userId para saber a quién pertenece el pedido para buscarlo
10     OrderResponse placeOrder(OrderRequest orderRequest, String userId); //
11
12     // List<OrderResponse> getAllOrders();
13
14     OrderResponse getOrderById(Long id); 1 usage 1 implementation ydochoadev
15
16     List<OrderResponse> getOrders(String userId, boolean isAdmin); 1 usage 1 implementation ydochoadev
17
18     void deleteOrder(Long id); 1 usage 1 implementation ydochoadev
19 }
```



Quedaría:



```
1 package com.ecommerce.order_service.service.impl;
2
3 > import ...
23
24 @Service & ydochoadev *
25 @RequiredArgsConstructor
26 @Slf4j
27 @RefreshScope
28 public class OrderServiceImpl implements OrderService {
29     private final OrderRepository orderRepository;
30     private final OrderMapper orderMapper;
31     // private final WebClient.Builder webClientBuilder;
32     private final InventoryClient inventoryClient;
33
34     @Value("${order.enabled:true}")
35     private boolean ordersEnabled;
36
37 @ public OrderResponse fallbackMethod(OrderRequest orderRequest, String userId, Throwable throwable) {
38     log.error("Circuit Breaker activado. Causa: {}", throwable.getMessage());
39
40     throw new RuntimeException("El servicio de Inventario no responde. Intentar más tarde");
41
42 }
43
44 @Override @usage new *
45 @Transactional
46 @CircuitBreaker(name = "inventory", fallbackMethod = "fallbackMethod")
47 @Retry(name = "inventory")
48 public OrderResponse placeOrder(OrderRequest orderRequest, String userId) {
```

```
19
20 import java.util.List;
21 import java.util.Map;
22
23 @RestController & ydochoadev *
24 @RequestMapping("/api/v1/order") // Versionado v1
25 @RequiredArgsConstructor
26 public class OrderController {
27     private final OrderService orderService;
28
29     @PostMapping & ydochoadev *
30     @ResponseStatus(HttpStatus.CREATED)
31     public OrderResponse placeOrder(@Valid @RequestBody OrderRequest orderRequest,
32                                     @AuthenticationPrincipal Jwt jwt) {
33         return orderService.placeOrder(orderRequest, jwt.getSubject());
34     }
35 }
```

La Letra Chica (The Catch)

⚠ Advertencia Arquitectónica

En arquitecturas síncronas (Spring MVC / Tomcat), el **TimeLimiter** cancela la espera del cliente, pero **NO mata el hilo del servidor**.

Esto puede generar "**Datos Fantasma**": El cliente recibe error, pero la operación se completa en segundo plano.

¿La solución definitiva?
Arquitectura Orientada a Eventos (RabbitMQ)